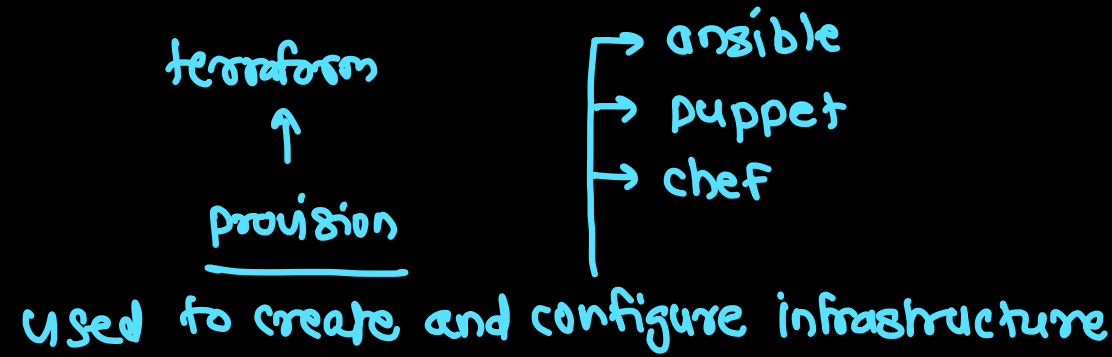
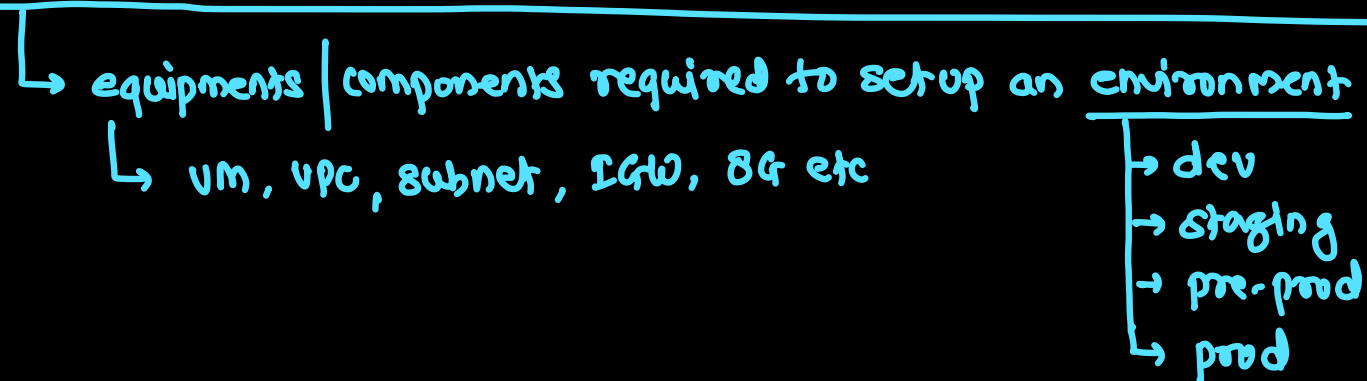




Infrastructure As Code (IaC)

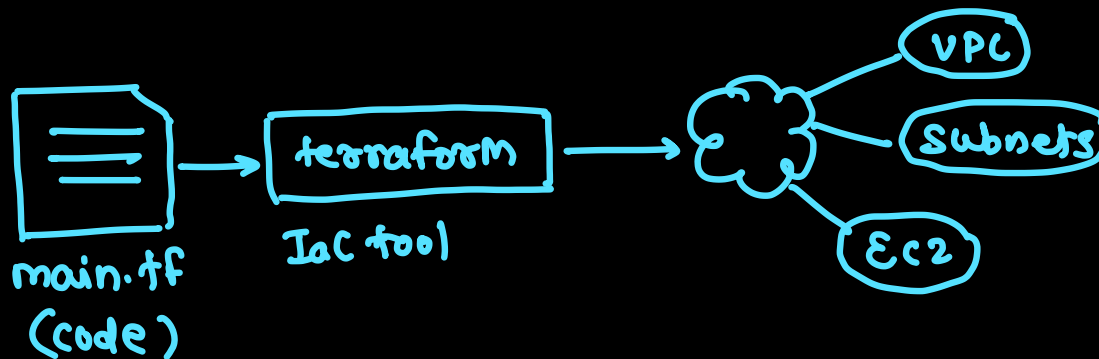


Introduction



Code

- Infrastructure as Code (IaC) is the practice of managing and provisioning computing infrastructure through machine-readable configuration files rather than manual processes or interactive configuration tools
- Instead of manually setting up servers, networks, databases, and other infrastructure components through a graphical interface or command-line instructions, you write code that describes exactly what infrastructure you need
- This code can then be executed to automatically create, modify, or destroy that infrastructure
- Think of it like building furniture: traditionally, you might assemble each piece manually, remembering which screws go where. With IaC, you'd have detailed assembly instructions (code) that anyone could follow to build exactly the same furniture every time, with no guesswork.



Why IaC Exists



■ Before IaC (traditional provisioning)

- ✓ Infrastructure was provisioned manually via UI/CLI
- ✓ Hard to track changes
- ✓ Error-prone (manual misconfigurations)
- ✓ No consistency between environments (DEV/QA/PROD drift)
- ✓ Scaling infrastructure required repetitive manual work
- ✓ Disaster recovery was slow (no automated infra recreation)

■ IaC solves this

- Automates provisioning → by using script / code
- Version controls your infrastructure
- Ensures consistency across environments
- Enables repeatability and idempotency
- Enables faster deployments and self-service provisioning

Benefits of IaC



- **Consistency**
 - Ensures that infrastructure across different environments is identical, reducing configuration drift
- **Scalability** → auto scaling
 - Easily replicate infrastructure for scaling up or down based on needs
- **Speed**
 - Automates the provisioning of complex setups, saving time compared to manual configuration
- **Cost-Efficiency**
 - Optimizes resource usage by scaling and automating teardown of unused resources
- **Collaboration**
 - Developers and operations teams can collaborate using version-controlled configuration files
- **Auditability**
 - Changes to infrastructure configurations are trackable through version control, enhancing security and compliance
- **Version Control** → code file can be track using any SCM / VCS tool
 - You can track changes to your infrastructure over time, see who changed what, and roll back if needed
- **Reusability**
 - Write the code once and deploy identical environments for development, testing, and production
- **Documentation**
 - The code itself serves as documentation of your infrastructure setup

Types of IaC



→ Setup environment

Type	Description	Examples
<u>Configuration Management (CM)</u> →	<u>Manages state of existing servers</u>	<u>Ansible, Puppet, Chef, SaltStack</u>
<u>Orchestration/Provisioning</u>	<u>Creates infrastructure from scratch</u>	<u>Terraform, CloudFormation, Pulumi</u>
<u>Container Orchestration IaC</u>	<u>Describes app infra using YAML</u>	<u>Kubernetes Manifests, Helm Charts, Kustomize</u>
<u>Policy as Code (PaC)</u>	<u>Enforces rules & governance</u>	<u>OPA, Sentinel</u>

IaC Tools Overview + Use Cases



■ Terraform

- Cloud-agnostic → AWS, GCP, Azure, Alibaba
- Declarative → instructions of provisioning resources
↓
HCL → block based
- Most widely used
- Best for provisioning infra
- State file manages drift

■ AWS CloudFormation

- AWS-native
- Declarative JSON/YAML
- Deep AWS integrations

■ Ansible

- Agentless
- Uses SSH
- Ideal for config management + ad-hoc automation

■ Puppet / Chef

- Long-standing CM tools
- Used in large enterprises
- Agent-based

■ Pulumi

- IaC using real programming languages (Python, TS, Go)

■ Kubernetes IaC

- Helm Chart = packaging
- Kustomize = YAML overlay
- Kubernetes YAML = declarative infra for containers

Pros and Cons



■ Pros

- Speed & automation
- Eliminates manual errors
- Consistent & repeatable
- Easy rollbacks
- Auditability with Git
- Collaboration through pull requests
- Scalability & reusability

■ Cons

- Learning curve
- State file management (Terraform) ***
- Misconfigurations can be automated (blast radius)
- Sensitive variable handling required
- Debugging can sometimes be difficult

Best Practices



■ Code Practices

- Keep IaC modular (Terraform modules, Ansible roles)
- Use variables & outputs
- Use Git branching strategies
- Enforce code reviews

■ Security Practices

- Store secrets in Vault, AWS Secrets Manager, Parameter Store
- Do not store credentials in files
- Enable encryption for state (Terraform backend S3 + DynamoDB lock)

■ Operational Practices

- Validate syntax using CI
- Run terraform plan always before apply
- Tag everything for cost tracking
- Keep environments isolated



Terraform

provisioning tool

What is Terraform



- HashiCorp Terraform is an infrastructure as code tool that lets you define both cloud and on-prem resources in human-readable configuration files that you can version, reuse, and share
- You can then use a consistent workflow to provision and manage all of your infrastructure throughout its lifecycle
- Terraform can manage low-level components like compute, storage, and networking resources, as well as high-level components like DNS entries and SaaS features



Terraform purpose

- Unify the view of resources using infrastructure as code
- Support the modern data center (IaaS, PaaS, SaaS)
- Expose a way for individuals and teams to safely and predictably change infrastructure
- Provide a workflow that is technology agnostic → code → .tf file
- Manage anything with an API provided by provider

Terraform Benefits



- Provides a high level abstraction of infrastructure (IaC)
- Allows for composition and combination
- Supports parallel management of resources
- Separates planning from execution (dry-run)

Characteristics



■ Declarative Configuration

- Instead of writing scripts or commands, you define the desired state of your infrastructure in .tf files, and Terraform handles the execution to achieve that state

■ Multi-Cloud Support

- Terraform works seamlessly with major cloud providers like AWS, Google Cloud, Azure, and more. It also supports on-premises solutions

■ Execution Plans

- Before making changes, Terraform generates a plan (terraform plan) to preview what actions it will perform, providing transparency and predictability

■ State Management ✨

- Terraform maintains a **state file** (terraform.tfstate) that records the current status of your infrastructure, ensuring accurate updates and management

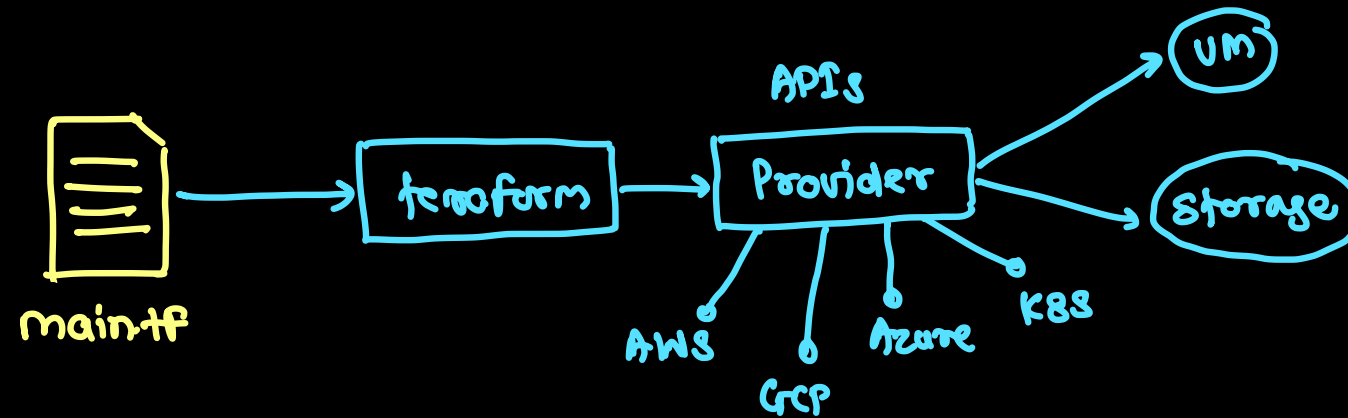
■ Modules for Reusability

- Infrastructure configurations can be modularized into reusable and shareable components, promoting consistency and scalability.

How it work?



- Terraform creates and manages resources on cloud platforms and other services through their application programming interfaces (APIs)
- Providers enable Terraform to work with virtually any platform or service with an accessible API
- HashiCorp and the Terraform community have already written thousands of providers to manage many different types of resources and services
- You can find all publicly available providers on the Terraform Registry, including Amazon Web Services (AWS), Azure, Google Cloud Platform (GCP), Kubernetes, Helm, GitHub, Splunk, DataDog, and many more



How it works ?



- The core Terraform workflow consists of three stages
- Write → *create tf file(s)*
 - You define resources, which may be across multiple cloud providers and services
 - For example, you might create a configuration to deploy an application on virtual machines in a Virtual Private Cloud (VPC) network with security groups and a load balancer
- Plan → *dry run*
 - Terraform creates an execution plan describing the infrastructure it will create, update, or destroy based on the existing infrastructure and your configuration
- Apply
 - On approval, Terraform performs the proposed operations in the correct order, respecting any resource dependencies
 - For example, if you update the properties of a VPC and change the number of virtual machines in that VPC, Terraform will recreate the VPC before scaling the virtual machines

Terraform Architecture



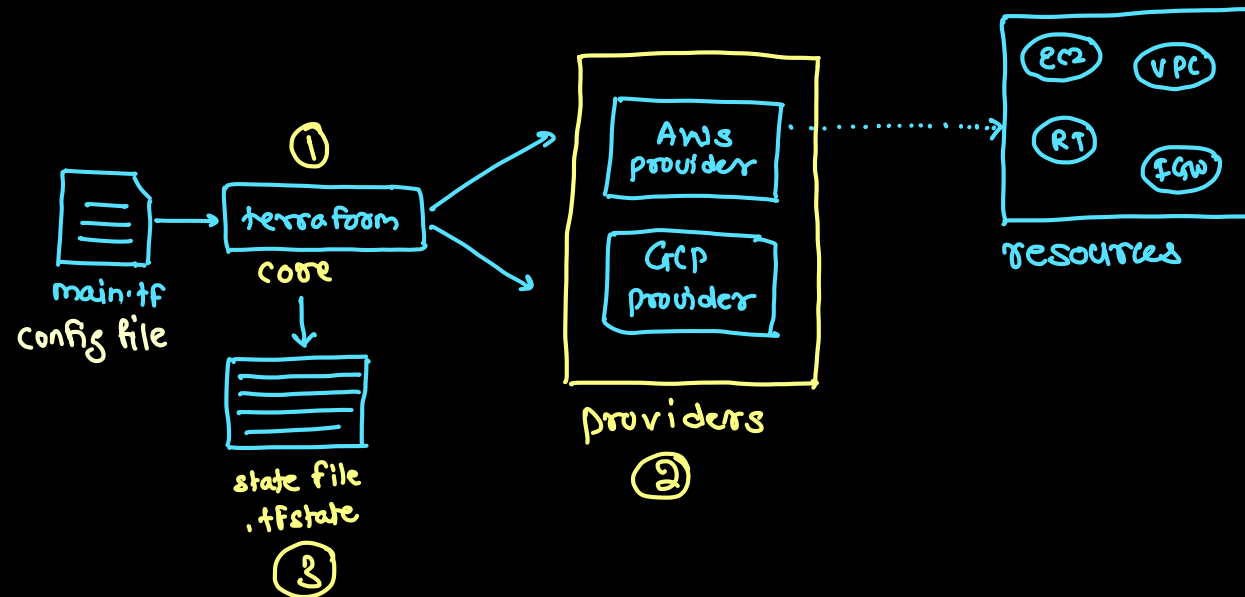
- Terraform architecture mainly consists of the following components
- **Terraform Core** *terraform command*
 - Terraform's core (also known as Terraform CLI) is built on a statically-compiled binary that's developed using the Go programming language
 - This binary is what generates the command line tool (CLI) known as "terraform," which serves as the primary interface for Terraform users
 - It is open source and can be accessed on the Terraform GitHub repository
- **Providers**
 - Terraform providers are modules that enable Terraform to communicate with a diverse range of services and resources, including but not limited to cloud providers, databases, and DNS services.
 - Each provider is responsible for defining the resources that Terraform can manage within a particular service and translating Terraform configurations into API calls that are specific to that service
 - Providers are available for numerous services and resources, including those developed by major cloud providers like AWS, Azure, and Google Cloud, as well as community-supported providers for various services
 - By utilizing providers, Terraform users can maintain their infrastructure in a consistent and reproducible manner, regardless of the underlying service or provider

Terraform Architecture



■ State file ✖✖✖

- The Terraform state file is an essential aspect of Terraform's functionality
- It is a JSON file that stores information about the resources that Terraform manages, as well as their current state and dependencies
- Terraform utilizes the state file to determine the changes that need to be made to the infrastructure when a new configuration is applied
- It also ensures that resources are not unnecessarily recreated across multiple runs of Terraform
- The state file can be kept locally on the machine running Terraform or remotely using a remote backend like Azure Storage Account or Amazon S3, or HashiCorp Consul
- It is crucial to safeguard the state file and maintain frequent backups since it contains sensitive information about the infrastructure being managed





Structure



Terraform Block



- In Terraform, a **block** is a basic unit of configuration that defines a specific component or behavior in an infrastructure setup
- Blocks are written in **HashiCorp Configuration Language (HCL)** and are enclosed by curly braces ({ })
- The **Terraform Block**, specifically, is a special configuration block that sets **global settings** for Terraform itself
- It is typically used to specify required versions of Terraform and providers, as well as backend configurations for state storage
- **Purpose of the Terraform Block**
 - **Provider Configuration:** Define and manage versions of required providers to maintain stability
 - **Version Control:** Ensure the correct version of Terraform is used
 - **State Management:** Configure how and where Terraform stores its state file, enabling collaboration and disaster recovery

```
terraform {  
  required_version = ">= 1.0.0"  
  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = ">= 3.0.0"  
    }  
  }  
}
```

Provider Block



- In Terraform, a provider block is a configuration block that specifies the provider you want to use to interact with cloud platforms, SaaS services, or other APIs
- Providers are responsible for managing the lifecycle of resources such as virtual machines, networks, or databases
- Providers act as plugins that Terraform uses to communicate with specific services like AWS, Azure, Google Cloud, Kubernetes, and many others
- **Purpose of the Provider Block**
 - **Authentication:** Configures credentials and other authentication details required to interact with the target platform
 - **Region and Environment Management:** Defines the scope and environment where resources will be deployed
 - **Multiple Instances:** Supports managing resources across multiple regions or accounts
 - **Customization:** Allows fine-tuning of provider behavior, such as timeouts, retries, or API endpoints

```
provider "aws" {  
    region = "us-west-2"  
    profile = "default"  
}
```

```
provider "azurerm" {  
    features {}  
    subscription_id = ""  
    tenant_id = ""  
}
```

```
provider "google" {  
    project = "project-a"  
    region = "us-central1"  
}
```

Resources Block



vm | vpc | RT | ...

- In Terraform, a **resource block** is a configuration block used to define and manage **individual infrastructure components**
- These components can be virtual machines, networks, databases, storage buckets, or any other resource supported by the provider
- Resource blocks are the most fundamental building blocks in Terraform configurations
- Each resource block specifies the type of resource and its settings
- **Structure of a Resource Block**
 - **<PROVIDER_TYPE>**
 - Specifies the type of resource and the provider managing it
 - Example: "aws_instance" for an AWS EC2 instance
 - **<RESOURCE_NAME>**
 - A user-defined name for the resource
 - This name is unique within the configuration and is used to reference the resource
 - Example: "web_server" for a web server EC2 instance
 - **Configuration Arguments**
 - Key-value pairs that define the properties of the resource

```
resource "<PROVIDER_TYPE>" "<RESOURCE_NAME>" {  
    # Configuration arguments  
}
```

```
resource "aws_instance" "web_server" {  
    ami = "ami-12345678"  
    instance_type = "t2.micro"  
}
```

Data Block



- A **data block** in Terraform is used to **fetch or query data from an existing resource** that is managed outside of the current Terraform configuration or by another process
- Unlike a resource block, which creates or manages infrastructure, a data block allows you to **read information** about resources without modifying them
- **Purpose of Data Blocks**
 - **Retrieve Existing Infrastructure Details:** Access details about resources created outside of Terraform, such as manually created AWS EC2 instances or S3 buckets
 - **Reference External Resources:** Query data from other Terraform-managed infrastructure, providers, or modules
 - **Dynamic Inputs:** Use queried data as inputs for other resources within configuration
- **Syntax**
 - **<PROVIDER_TYPE>**
 - Specifies the type of data to query and the provider that offers it
 - Example: `aws_ami`, `azurerm_resource_group`, `google_project`
 - **<NAME>**
 - A logical name to reference the data source in the configuration
 - **Query Arguments**
 - Parameters to filter or identify the data being fetched

```
data "<PROVIDER_TYPE>" "<NAME>" {  
    # Query arguments  
}
```

```
data "aws_vpc" "default" {  
    default = true  
}  
resource "aws_subnet" "example" {  
    vpc_id = data.aws_vpc.default.id  
}
```

Variable Block



- A **variable block** in Terraform is used to define **input variables** that make configurations dynamic, reusable, and flexible
- Instead of hardcoding values directly in Terraform configuration files, variables allow you to define them externally and pass them into the configuration at runtime
- This enables better manageability, especially when working with different environments (e.g., development, staging, production)
- **Structure of a Variable Block**
 - **<VARIABLE_NAME>**
 - The name of the variable, used to reference it in other parts of the configuration
 - **Optional Arguments**
 - Define properties like default values, descriptions, type constraints, and more
- **Components of a Variable Block**
 - **Name**
 - A unique identifier used to reference the variable throughout the configuration.
 - **Type (Optional)**
 - **string**: Single-line text (e.g., "t2.micro")
 - **number**: Numeric values (e.g., 10, 3.14)
 - **bool**: Boolean values (true or false)
 - **list**: A list of values (e.g., ["a", "b", "c"])
 - **map**: A key-value pair (e.g., {key1 = "value1", key2 = "value2"})
 - **object**: Complex structures with specific attribute types
 - **any**: Accepts any data type

```
variable "<VARIABLE_NAME>" {  
    # Optional arguments  
}
```

```
variable "region" {  
    type = string  
    default = "us-east-1"  
}  
provider "aws" {  
    region = var.region  
}
```

Output Block



- An **output block** in Terraform is used to display information about your infrastructure after the configuration has been applied
- Outputs can provide useful data, such as resource attributes, that you may need for debugging, referencing, or passing to other systems or modules
- **Structure of an Output Block**
 - **<OUTPUT_NAME>**
 - The name of the output variable. It should be unique within the configuration
 - **Value**
 - The expression or data to be returned. This often references attributes of resources or variables
- **Purpose of Output Blocks**
 - **Informative Outputs**
 - Provide helpful information after a terraform apply or terraform plan.
 - Example: IP addresses, DNS names, or resource IDs.
 - **Debugging**
 - Use outputs to confirm that resources were created with the correct attributes.
 - **Passing Data Between Modules**
 - Share resource attributes between modules in a Terraform configuration.
 - **Integration with Other Tools**
 - Outputs can be used in scripts or tools to automate further actions.

```
output "< OUTPUT _NAME>" {  
    value = <EXPRESSION>  
}
```

Commands



- **terraform init**

- Terraform init command initializes a Terraform working directory by downloading and installing any required plugins and dependencies
- It should be run before any other Terraform commands

- **terraform validate**

- The validate command performs precisely what its name implies
- It ensures that the code is internally coherent and examines it for syntax mistakes
- Only the configuration files (*.tf) in the active working directory are examined
- You must provide the -a recursive flag if you want to validate files inside of folders (for example, if you have a module/ directory)

- **terraform apply**

- Terraform apply command applies the changes defined in the configuration to your infrastructure
- It creates or updates the resources according to the configuration, and it also prompts you to confirm the changes before applying them

Commands



- **terraform destroy**
 - Terraform destroy command will destroy all the resources created by Terraform in the current working directory
 - It is a useful command for tearing down your infrastructure when you no longer need it
- **terraform import**
 - Imports an existing resource into the Terraform state, allowing it to be managed by Terraform
- **terraform console**
 - Opens an interactive console for evaluating expressions in the Terraform configuration
- **terraform refresh**
 - This command updates the state of your infrastructure to reflect the actual state of your resources
 - It is useful when you want to ensure that your Terraform state is in sync with the actual state of your infrastructure